

Student

Last time we covered CTEs as a readability tool. You mentioned recursive CTEs unlock a whole different class of problems. What kind of problems can a normal CTE not solve?

ChatGPT • AI Assistant

CONCEPT CARD

Standard SQL has no concept of a loop. A regular JOIN, no matter how many times you write it, only traverses a **fixed, known number of relationships**. Two specific problem shapes break this:

1. **'Find every employee under this manager, at any depth'** - you do not know in advance whether the chain is 2 levels deep or 12 levels deep. A fixed number of JOINS cannot handle an unknown depth.
2. **'Expand a bill-of-materials tree'** - a product is made of sub-assemblies, which are made of sub-assemblies, which are made of raw parts. The number of levels varies per product.

Recursive CTEs solve exactly this - they let a CTE reference itself, repeating the same JOIN logic over and over until no new rows are produced, regardless of how deep the hierarchy actually goes.

CONCEPT CARD**The anatomy of a recursive CTE:**

sql – Code Example

```
WITH RECURSIVE cte_name AS (  
    -- ANCHOR MEMBER: the base case. Runs exactly ONCE.  
    SELECT ...  
    FROM   some_table  
    WHERE  starting_condition  
  
    UNION ALL  
  
    -- RECURSIVE MEMBER: references cte_name itself.  
    -- Runs REPEATEDLY, each time operating on the rows  
    -- the PREVIOUS iteration produced – not the whole table.
```

```

SELECT ...
FROM   some_table t
JOIN   cte_name c ON t.parent_col = c.id_col
)
SELECT * FROM cte_name;

```

Two parts joined by **UNION ALL** :

- **Anchor member** - the starting point. Seeds the recursion with the first batch of rows.
- **Recursive member** - references the CTE by its own name. Each execution operates only on the rows the **immediately preceding** iteration produced, joins them to the base table again, and appends any new matching rows.

🔗 ANALOGY

Think of the anchor as lighting the first domino, and the recursive member as the rule 'each falling domino knocks over the next one it touches.' The rule is applied over and over - not because you wrote it multiple times, but because each step naturally produces the input for the next step. The chain stops the moment a domino falls with no other domino in front of it.

CONCEPT CARD

The execution model, step by step:

code – Code Example

```

Step 0: Run the ANCHOR member.           → produces Result Set R0
Step 1: Run the RECURSIVE member
        using R0 as the input "cte_name". → produces Result Set R1
Step 2: Run the RECURSIVE member
        using R1 (NOT R0!) as input.       → produces Result Set R2
Step 3: Run the RECURSIVE member
        using R2 as input.                 → produces Result Set R3
...
Stop:   The recursive member returns ZERO rows
        → recursion halts.
Final result = R0 UNION ALL R1 UNION ALL R2 UNION ALL R3 ...

```

Each iteration only sees the **previous** iteration's output - never the cumulative total, and never the original anchor again after Step 1. This is the single most important and most commonly misunderstood detail of recursive CTEs.

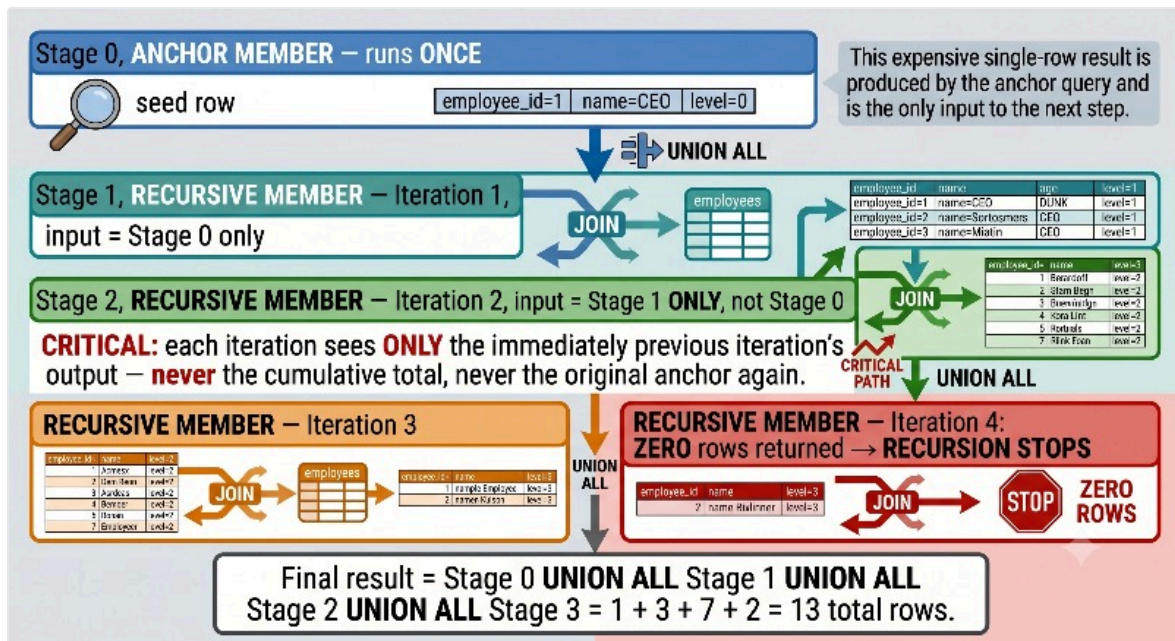
What schema are we using?

ChatGPT • AI Assistant

CONCEPT CARD

Two small hierarchical tables for this topic, plus the e-commerce schema for the date-sequence problem:

- `employees(employee_id, name, manager_id, department, salary)` - `manager_id` is NULL for the CEO
- `categories(category_id, category_name, parent_id)` - `parent_id` is NULL for top-level categories
- `orders(order_id, customer_id, order_date, total_amount, status)` - used in the practice problems



WALKING DOWN A HIERARCHY — THE EMPLOYEE REPORTING CHAIN

ChatGPT • AI Assistant

An org chart tree diagram with 8 employee nodes arranged hierarchically. Root node (top, gold, label 'Carol — Level 1'). Three child nodes directly below connected by lines: 'Alice — Level 2', 'Bob — Level 2', 'Dave — Level 2' — all shaded the same teal color representing Iteration 1's output. Below Alice, two child nodes: 'Eve — Level 3', 'Frank — Level 3', shaded green representing Iteration 2's output. Below Dave, one child node: 'Grace — Level 3', also shaded green (same iteration as Eve and Frank, since both branches advance together). Bob has no children — shown with a dashed empty box below labeled 'No direct reports — this branch terminates here.' Below Eve, one child node: 'Henry — Level 4', shaded orange representing Iteration 3's output. Below Henry, an empty dashed box labeled 'No direct reports — recursion stops for this branch in Iteration 4.' A color legend on the right maps: gold=Anchor (Iteration 0), teal=Iteration 1, green=Iteration 2, orange=Iteration 3. A note at the bottom reads: 'All branches advance in lockstep — recursion fully stops only when EVERY active branch in an iteration produces zero new rows.'

Employee Reporting Chain — Org Chart with Iteration Levels Marked

Student

Let's build the classic example. I want every employee under a given CEO, at any depth, with a level column showing how deep each one is.

ChatGPT • AI Assistant

CONCEPT CARD

Using `employees(employee_id, name, manager_id, department, salary)` :

sql – Code Example

```
WITH RECURSIVE reporting_chain AS (  
  -- ANCHOR: start at the CEO (employee_id = 1, manager_id IS NULL)  
  SELECT employee_id,  
         name,  
         manager_id,  
         department,  
         1 AS level           -- the CEO is level 1  
  FROM   employees  
  WHERE  employee_id = 1  
  
  UNION ALL  
  
  -- RECURSIVE MEMBER: find direct reports of everyone  
  -- found in the PREVIOUS iteration  
  SELECT e.employee_id,  
         e.name,  
         e.manager_id,  
         e.department,  
         rc.level + 1        -- one level deeper than the  
  FROM   employees e  
  JOIN   reporting_chain rc ON e.manager_id = rc.employee_id  
  WHERE  e.manager_id IS NOT NULL  
)
```

```
FROM    employees e
JOIN    reporting_chain rc ON e.manager_id = rc.employee_id
)
SELECT  employee_id, name, manager_id, department, level
FROM    reporting_chain
ORDER BY level, employee_id;
```

EXAMPLE CARD

Tracing through the first three iterations by hand:

Assume this data:

code – Code Example				
employee_id	name	manager_id	department	
1	Carol	NULL	Executive	← CEO
2	Alice	1	Engineering	
3	Bob	1	Engineering	
4	Dave	1	Sales	
5	Eve	2	Engineering	
6	Frank	2	Engineering	
7	Grace	4	Sales	
8	Henry	5	Engineering	

Anchor (Step 0):

```
code – Code Example

WHERE employee_id = 1 → produces:
employee_id=1, name=Carol, manager_id=NULL, level=1
```

Result of Step 0 (R0): **{Carol, level 1}** - 1 row.

Recursive Iteration 1 (uses R0 as input):

```
code – Code Example

JOIN employees e ON e.manager_id = rc.employee_id
-- rc currently only contains Carol (employee_id=1)
-- find every employee whose manager_id = 1
```

Result R1: **{Alice level 2, Bob level 2, Dave level 2}** - 3 rows. (Carol's three direct reports.)

Recursive Iteration 2 (uses R1 as input - NOT R0!):

code – Code Example

```
-- rc now only contains Alice, Bob, Dave (the output of the LAST
iteration)
-- find every employee whose manager_id IN (2, 3, 4)
```

Result R2: **{Eve level 3, Frank level 3, Grace level 3}** - 3 rows. (Eve and Frank report to Alice; Grace reports to Dave; nobody reports to Bob.)

Recursive Iteration 3 (uses R2 as input):

code – Code Example

```
-- rc now only contains Eve, Frank, Grace
-- find every employee whose manager_id IN (5, 6, 7)
```

Result R3: **{Henry level 4}** - 1 row. (Henry reports to Eve.)

Recursive Iteration 4 (uses R3 as input):

code – Code Example

```
-- rc now only contains Henry
-- find every employee whose manager_id = 8 → NONE EXIST
```

Result: **0 rows** → recursion stops.

Final combined result (R0 ∪ R1 ∪ R2 ∪ R3):

code – Code Example

employee_id	name	level
1	Carol	1
2	Alice	2
3	Bob	2
4	Dave	2
5	Eve	3
6	Frank	3
7	Grace	3
8	Henry	4

Notice Bob (level 2) produced zero rows in his own iteration because nobody reports to him - but that does not stop the overall recursion, because Eve, Frank, and Dave's chains were still active. Recursion only fully stops when an **entire iteration** - across all currently-active branches - produces zero new rows.

⚠ WARNING

A very common misconception: people assume the recursive member re-joins against the **whole accumulated result so far**. It does not.

It joins only against the rows produced by the **immediately preceding** iteration. This is precisely why a manager who already appears in the result (like Carol) is never re-matched again on a later iteration - she is not part of the input to Iteration 2 or beyond, only the employees newly discovered in Iteration 1 are.

🔗 INTERVIEW CARD

Frequency: Very High

Question: Explain step by step what happens when a recursive CTE executes. What exactly does the recursive member operate on during each iteration?

Expected Answer: The anchor member runs first and produces an initial result set - call it R0. The recursive member then runs once using R0 as the contents of the CTE name, producing R1.

Critically, the next execution of the recursive member uses R1 - not R0, and not the cumulative total - as the contents of the CTE name, producing R2. This repeats: each iteration's output becomes the next iteration's sole input.

The process halts the moment an iteration of the recursive member returns zero rows. The final result returned to the user is the UNION ALL of every iteration's output, R0 through the last non-empty iteration.

Follow-up: Why does a manager like Bob, who has no direct reports, not stop the whole recursion if he appears in the result? (Answer: Recursion halts only when an entire iteration produces zero rows across all currently active branches - not when one individual row produces no children.)

As long as at least one other row from that iteration still has matching children in the next pass, the recursion continues for those branches; rows with no children simply contribute nothing further.)

❓ PRACTICE QUIZ

1. In the trace above, what is the input to Recursive Iteration 2 - R0, R1, or the cumulative total so far?

2. Why does recursion stop after Iteration 4 in the example?
3. If Henry had a direct report named Ivy, what level would Ivy be assigned?
4. What does the level column represent and how is it computed in the recursive member?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 **ANSWER KEY**

1. R1 - the output of the immediately preceding iteration (Alice, Bob, Dave), not R0 (Carol) and not the cumulative total.
2. Because Iteration 4 tried to find employees whose manager_id = 8 (Henry) and found none - zero rows produced means the recursion halts.
3. Level 5 - one level deeper than Henry (level 4), since the recursive member always computes `rc.level + 1`.
4. The level column represents the depth in the hierarchy from the anchor (CEO = level 1). It is computed in the recursive member as `rc.level + 1`, taking the parent's level from the previous iteration and adding one.

❓ **CHECKPOINT QUESTION**

Using the same employees table, write a recursive CTE that returns every employee under Alice (employee_id = 2) specifically - not the whole company - with a level column starting at 1 for Alice herself. Based on the sample data above, list exactly which employees and levels you expect in the result.

BUILDING A FULL PATH STRING — CATEGORY TREE TRAVERSAL



ChatGPT • AI Assistant

A tree diagram with category nodes connected to a parallel path-string accumulator panel. Top node (gold, label 'Electronics — full_path = Electronics, depth=1'). Below it, a child node (teal, label 'Computers — full_path = Electronics > Computers, depth=2') with an arrow from the parent showing 'cp.full_path (Electronics) || > || Computers'. Below Computers, two child nodes (green, label 'Laptops — full_path = Electronics > Computers > Laptops, depth=3' and 'Desktops — full_path = Electronics > Computers > Desktops, depth=3'), each with an arrow showing the concatenation step appending their own name to the inherited path. A separate small tree on the right shows 'Fitness — full_path = Fitness, depth=1' with one child 'Yoga — full_path = Fitness > Yoga, depth=2'. A side annotation box reads: 'Each iteration receives the PARENT path already built, and appends exactly one more segment — the string grows by one level per recursive step, mirroring the depth counter.' Clean tree layout with path-string callouts attached to each node showing the exact concatenation expression used to produce it.

Category Tree Path Building — String Concatenation Across Iterations

Student

Show me a second example - traversing a category tree and building the full path as a readable string, like 'Electronics > Computers > Laptops'.

ChatGPT • AI Assistant

CONCEPT CARD

Using `categories(category_id, category_name, parent_id)` , where `parent_id` is NULL for top-level (root) categories:

sql – Code Example

```
WITH RECURSIVE category_path AS (  
    -- ANCHOR: every TOP-LEVEL category (no parent)  
    SELECT category_id,  
           category_name,  
           parent_id,  
           category_name::TEXT AS full_path,    -- path starts as just  
the name  
           1 AS depth  
    FROM categories  
    WHERE parent_id IS NULL  
  
    UNION ALL  
  
    -- RECURSIVE MEMBER: find children of categories found so far,  
    -- and APPEND their name onto the parent's already-built path  
    SELECT c.category_id,  
           c.category_name,  
           c.parent_id,  
           cp.full_path || ' > ' || c.category_name AS full_path,
```

```
        cp.depth + 1                                AS depth
FROM    categories c
JOIN    category_path cp ON c.parent_id = cp.category_id
)
SELECT category_id, category_name, full_path, depth
FROM    category_path
ORDER BY full_path;
```

EXAMPLE CARD

Sample data and resulting paths:

code – Code Example

category_id	category_name	parent_id
1	Electronics	NULL
2	Computers	1
3	Laptops	2
4	Desktops	2
5	Fitness	NULL
6	Yoga	5

Output:

code – Code Example

category_id	category_name	full_path	depth
1	Electronics	Electronics	1
2	Computers	Electronics > Computers	2
3	Laptops	Electronics > Computers > Laptops	3
4	Desktops	Electronics > Computers > Desktops	3
5	Fitness	Fitness	1
6	Yoga	Fitness > Yoga	2

The key mechanism: `cp.full_path || ' > ' || c.category_name` takes the **already-built path string from the parent's row** (carried forward from the previous iteration) and appends the current row's name to it. Each iteration extends every active path by exactly one more segment.

CONCEPT CARD

Finding all descendants of a SPECIFIC category (not the whole tree):

sql – Code Example

```
WITH RECURSIVE category_descendants AS (  
    -- ANCHOR: start at ONE specific category, e.g. "Electronics"  
    (id=1)  
    SELECT category_id, category_name, parent_id, 1 AS depth  
    FROM categories  
    WHERE category_id = 1  
  
    UNION ALL  
  
    SELECT c.category_id, c.category_name, c.parent_id, cd.depth + 1  
    FROM categories c  
    JOIN category_descendants cd ON c.parent_id = cd.category_id  
)  
SELECT * FROM category_descendants;  
-- Returns: Electronics, Computers, Laptops, Desktops (everything  
UNDER Electronics)  
-- Does NOT return Fitness or Yoga – they are outside this category's  
subtree
```

The only difference from the full-tree version is the anchor's WHERE clause: filtering to one starting category instead of `parent_id IS NULL` for all roots.

 INTERVIEW CARD

Frequency: High

Question: How would you build a breadcrumb path string for every category in a tree using a recursive CTE?

Expected Answer: The anchor member selects all root categories (*parentid IS NULL*) and initialises the path as just that category's own name. The recursive member joins the categories table to the CTE on *parentid = cte.categoryid*, and builds the new path by concatenating the parent's already-accumulated path (carried in from the previous iteration) with a separator and the current category's name - for example, ``cp.fullpath || ' > ' || c.category_name``.

Because each iteration only has access to the path string built so far, the concatenation naturally grows by exactly one segment per level, producing a full breadcrumb from root to the current node by the time recursion reaches it.

Follow-up: How would you modify the query to find only the descendants of one specific category rather than the entire tree? (Answer: Change the anchor's WHERE clause from `parent_id IS NULL` to `category_id = <specific_id>` - everything else in the query, including the recursive member, stays identical.

This restricts the starting point of the recursion to one subtree instead of seeding it with every root.)

🔗 PRACTICE QUIZ

1. In the recursive member, where does the value of `cp.full_path` for the parent come from?
2. What changes if you want descendants of one category instead of the whole tree?
3. What does the depth column tell you in this example?
4. If a category has no children, does it still appear once in the final result?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔗 ANSWER KEY

1. It comes from the row produced for that parent in a previous iteration - carried forward through the CTE exactly as it was built, then extended with `|| ' > ' || c.category_name` for the current row.
2. Only the anchor member's WHERE clause changes - from `WHERE parent_id IS NULL` (all roots) to `WHERE category_id = <specific_id>` (one starting node). The recursive member is unchanged.
3. Depth tells you how many levels deep the category is from its root - the anchor row is depth 1, and each recursive step adds 1.
4. Yes - every category is produced exactly once, either by the anchor (if it is a root) or by exactly one iteration of the recursive member (if it has a parent).

Having no children only means it produces no further rows in subsequent iterations - it does not prevent it from appearing once itself.

🔗 CHECKPOINT QUESTION

Write a recursive CTE on the categories table that returns, for every category, not the full path string but the **root category name** it ultimately belongs to (e.g., Laptops → Electronics, Yoga → Fitness). Hint: instead of growing a path string, carry the root's name forward unchanged through every iteration.

PREVENTING INFINITE LOOPS — DEPTH GUARDS AND CYCLE DETECTION

A diagram showing two nodes locked in a cycle. Node A (red, label 'Eve — employee_id=5, manager_id=8') has an arrow pointing to Node B. Node B (red, label 'Henry — employee_id=8, manager_id=5') has an arrow pointing back to Node A, forming a closed loop with both arrows highlighted in bright red and a repeating circular arrow icon in the center labeled 'INFINITE LOOP.' Below the cycle diagram, a timeline shows iteration attempts: 'Iteration 1: finds Henry (reports to Eve). Iteration 2: finds Eve (reports to Henry). Iteration 3: finds Henry again. Iteration 4: finds Eve again...' continuing with an ellipsis and a memory usage bar climbing steadily upward labeled 'Memory consumption increasing — never reaches zero new rows.' On the right side, a green 'WITH SAFEGUARD' panel shows the same cycle but with a depth guard applied: 'Iteration 1 through 99: keeps alternating Eve/Henry. Iteration 100: WHERE level < 100 blocks further recursion — query terminates with a truncated (but bounded) result instead of hanging forever.' Clean before/after comparison, red cycle on the left labeled 'CRASH RISK', green bounded version on the right labeled 'SAFE — bounded by depth guard.'

Cycle in Hierarchical Data — Infinite Loop Without a Safeguard

Student

What happens if the data has a cycle - like a manager_id that accidentally points back down the chain, creating a loop?

⚠️ WARNING

This is the single most dangerous failure mode of recursive CTEs: **an infinite loop that can consume all available memory and crash the session or the server.**

sql — Code Example

```
-- A corrupted employees table where Eve's manager_id was
-- accidentally set to point to Henry, who reports to Eve:
employee_id | name | manager_id
-----|-----|-----
5           | Eve  | 8           -- Eve now reports to Henry ...
8           | Henry | 5          -- ... who reports to Eve. CYCLE!
```

With this cycle present, the recursive member never produces zero rows - it keeps alternating between finding Henry (because Eve is in the result) and finding Eve (because Henry is in the result), forever. **Without a safeguard, this query never terminates.**

CONCEPT CARD

Safeguard 1 - A depth guard (the simplest and most common protection):

sql — Code Example

```

WITH RECURSIVE reporting_chain AS (
    SELECT employee_id, name, manager_id, 1 AS level
    FROM   employees
    WHERE  employee_id = 1

    UNION ALL

    SELECT e.employee_id, e.name, e.manager_id, rc.level + 1
    FROM   employees e
    JOIN   reporting_chain rc ON e.manager_id = rc.employee_id
    WHERE  rc.level < 100      -- HARD STOP: never recurse past 100
levels
)
SELECT * FROM reporting_chain;

```

The **WHERE rc.level < 100** clause is checked **inside the recursive member** - once any branch reaches level 100, that branch stops contributing new rows, regardless of whether real data or a cycle is driving it. Choose a depth limit comfortably larger than any legitimate hierarchy you expect (a 100-level org chart is implausible; a 10-12 level limit is often safer and more realistic for a company hierarchy), and the query simply truncates if something is wrong rather than running forever.

CONCEPT CARD

Safeguard 2 - A visited-path check (catches cycles directly, not just by truncating depth):

sql – Code Example

```

WITH RECURSIVE reporting_chain AS (
    SELECT employee_id,
           name,
           manager_id,
           1                                     AS level,
           ARRAY[employee_id]                  AS visited_path
    FROM   employees
    WHERE  employee_id = 1

    UNION ALL

    SELECT e.employee_id,
           e.name,
           e.manager_id,
           rc.level + 1,
           rc.visited_path || e.employee_id     -- append current id

```

```

to path
    FROM    employees e
    JOIN    reporting_chain rc ON e.manager_id = rc.employee_id
    WHERE   NOT (e.employee_id = ANY(rc.visited_path)) -- stop if
already visited
)
SELECT employee_id, name, level, visited_path
FROM    reporting_chain;

```

This carries an array of every `employee_id` already seen along the current branch's path. The `WHERE NOT (e.employee_id = ANY(rc.visited_path))` condition explicitly refuses to revisit any node already present in that specific branch's history - this catches a cycle on the very first repeated node, rather than merely truncating after an arbitrary depth limit.

It is more precise but slightly more expensive (array operations per row) than a simple depth guard.

⚠ WARNING

Always include at least ONE of these two safeguards in any production recursive CTE, even when you believe the data cannot contain cycles. Data quality issues happen - a single bad row (someone's *managerid accidentally set to their own employeeid*, or a self-referencing *parent_id*) is enough to hang a query indefinitely and potentially exhaust server memory, affecting other users on a shared database.

⚠ COMMON PITFALL / MISTAKE

✗ Trusting the data and skipping any safeguard:

sql – Code Example

```

WITH RECURSIVE chain AS (
    SELECT employee_id, manager_id, 1 AS level FROM employees WHERE
employee_id = 1
    UNION ALL
    SELECT e.employee_id, e.manager_id, c.level + 1
    FROM employees e JOIN chain c ON e.manager_id = c.employee_id
    -- NO depth guard, NO visited-path check
)
SELECT * FROM chain;
-- If even ONE row has a cyclic manager_id, this runs forever.

```

✓ Always guard, even on data you believe is clean:

sql – Code Example

```
WITH RECURSIVE chain AS (  
    SELECT employee_id, manager_id, 1 AS level FROM employees WHERE  
    employee_id = 1  
    UNION ALL  
    SELECT e.employee_id, e.manager_id, c.level + 1  
    FROM employees e JOIN chain c ON e.manager_id = c.employee_id  
    WHERE c.level < 50    -- minimal safety net, cheap insurance  
)  
SELECT * FROM chain;
```

INTERVIEW CARD

Frequency: Very High

Question: What safety measures must you include in a production recursive CTE, and why?

Expected Answer: Always include either a depth guard or a visited-path check, because cyclic data - even a single corrupted row - causes a recursive CTE to run forever, potentially exhausting server memory and affecting other database users. A depth guard, written as a WHERE clause inside the recursive member like **WHERE rc.level < 100**, is the simplest protection: it caps the number of iterations regardless of why the recursion keeps producing rows.

A visited-path check carries an array of already-seen keys through each iteration and explicitly refuses to revisit any key already in that array, catching the exact point of a cycle rather than just truncating after an arbitrary depth. Production code should include at least one of these even when the data is believed to be clean, since this single safeguard prevents an entire class of catastrophic, hard-to-diagnose hangs.

Follow-up: Why is a visited-path check more precise than a depth guard, and what is its trade-off? (Answer: A depth guard stops recursion after N levels regardless of cause - it could truncate a legitimately deep but cycle-free hierarchy just as easily as it stops an actual cycle, and it does not tell you WHERE a cycle occurred.)

A visited-path check detects the exact repeated node causing a cycle, which is more diagnostically useful, but it requires carrying and comparing an array on every row of every iteration, which is more memory and CPU intensive than a single integer comparison.)

PRACTICE QUIZ

1. What happens to a recursive CTE if the underlying data contains a cycle and no safeguard is present?

2. Where exactly does a depth guard's WHERE clause go - in the anchor member or the recursive member?
3. What does a visited-path check carry forward through each iteration?
4. Which safeguard is cheaper to evaluate per row: a depth guard or a visited-path check?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 **ANSWER KEY**

1. The recursive member never produces zero rows - it loops forever, consuming increasing memory and potentially hanging or crashing the session or the database server.
2. In the recursive member - it is evaluated as part of the JOIN condition during every recursive iteration, since the anchor member only ever runs once and has no concept of 'level' increasing yet.
3. An array (or similar collection) of every key already visited along that specific branch's path so far - e.g., `ARRAY[employee_id]` extended with `|| e.employee_id` at each step.
4. A depth guard - it is a single integer comparison (`level < 100`), whereas a visited-path check requires an array containment test (`= ANY(array)`) which is more expensive per row.

❓ **CHECKPOINT QUESTION**

A junior engineer writes a recursive CTE on the categories table to build full paths, but forgets any safeguard. A data entry error causes *categoryid 12's parentid* to be set to *categoryid 45*, while *categoryid 45's parentid* is set to *categoryid 12*.

What happens when the query runs, and write the corrected version using a depth guard.

PERFORMANCE REALITY AND WHEN A CLOSURE TABLE IS THE BETTER DESIGN

A two-panel architecture diagram. Left panel (orange background, label 'RECURSIVE CTE AT READ TIME'): shows a user request icon triggering a 'WITH RECURSIVE' query that performs multiple chained JOIN iterations (shown as 5 stacked JOIN boxes) against the categories table, taking a clock icon showing '40ms per request.' Annotation: 'Recomputes the full traversal from scratch on EVERY single request.' A volume indicator shows '1,000,000 requests/day × 40ms = significant cumulative cost.' Right panel (green background, label 'CLOSURE TABLE ARCHITECTURE'): shows two separate flows. Flow 1 (top, infrequent): a 'Tree Changed' trigger icon leads to a batch job box labeled 'Rebuild closure table — runs recursive CTE ONCE, writes results to category_closure table.' Flow 2 (bottom, frequent): a user request icon hits a simple indexed JOIN against the pre-built category_closure table, taking a clock icon showing '0.5ms per request.' Annotation: 'Reads a pre-computed flat table — no recursion at request time at all.' Volume indicator shows '1,000,000 requests/day × 0.5ms = 80x cost reduction.' A center divider reads: 'Move the recursive computation from the hot read path to an infrequent batch job triggered only when the hierarchy actually changes.'

Recursive CTE at Read Time vs Closure Table — Architecture Comparison

Student

Are recursive CTEs always the right tool, or are there cases where a different design is better for production?

ChatGPT • AI Assistant

CONCEPT CARD

Recursive CTEs are powerful, but they have real performance costs that grow with hierarchy depth and breadth:

Why recursive CTEs can be expensive:

code — Code Example

- Each iteration is a full JOIN operation – the database executes the recursive member's JOIN once per LEVEL of the hierarchy, not once total.
- A hierarchy 15 levels deep means 15 separate JOIN executions chained together.
- A WIDE hierarchy (each node has many children) multiplies the row count exponentially with depth if not carefully filtered.
- The optimizer has very limited ability to optimize ACROSS iterations
- each step is planned somewhat independently.
- No native index can speed up 'the result of iteration N' the way an index speeds up a normal WHERE clause on a static table.

EXAMPLE CARD

When this becomes a real problem:

sql – Code Example

```
-- An e-commerce category tree with 8 levels and thousands of
categories,
-- queried on every single product page load to build a breadcrumb:
WITH RECURSIVE category_path AS ( ... )
SELECT full_path FROM category_path WHERE category_id = 4521;
-- If this runs on EVERY page view (millions of times per day),
-- recomputing the recursive traversal from scratch every single time
-- is wasteful – the category tree itself rarely changes.
```

CONCEPT CARD

The production alternative: a closure table (also called a path table)

A closure table precomputes and stores every ancestor-descendant relationship as a flat, indexable table - built once (or refreshed periodically), then queried with simple, fast, indexed lookups instead of a recursive CTE every time.

sql – Code Example

```
-- Closure table: one row per (ancestor, descendant) pair, with depth
CREATE TABLE category_closure (
    ancestor_id    INT NOT NULL,
    descendant_id  INT NOT NULL,
    depth          INT NOT NULL,    -- 0 = self, 1 = direct child, etc.
    PRIMARY KEY (ancestor_id, descendant_id)
);

-- Populate it ONCE using a recursive CTE (or a batch job)
INSERT INTO category_closure (ancestor_id, descendant_id, depth)
WITH RECURSIVE expand AS (
    SELECT category_id AS ancestor_id, category_id AS descendant_id, 0
    AS depth
    FROM    categories
    UNION ALL
    SELECT e.ancestor_id, c.category_id, e.depth + 1
    FROM    expand e
    JOIN    categories c ON c.parent_id = e.descendant_id
)
SELECT ancestor_id, descendant_id, depth FROM expand;

CREATE INDEX idx_closure_descendant ON
category_closure(descendant_id);
CREATE INDEX idx_closure_ancestor  ON category_closure(ancestor_id);
```

```
-- Now the breadcrumb query becomes a FAST, INDEXED, NON-RECURSIVE
lookup:
SELECT c.category_name
FROM   category_closure cl
JOIN   categories c ON cl.ancestor_id = c.category_id
WHERE  cl.descendant_id = 4521
ORDER BY cl.depth DESC;           -- furthest ancestor (root) first
-- This is a simple indexed JOIN – no recursion at read time at all.
```

Alternative: a materialized path column (simpler, sufficient for many cases)

sql – Code Example

```
-- Store the path directly on the row, maintained on insert/update
ALTER TABLE categories ADD COLUMN materialized_path TEXT;

-- e.g., category_id=3 (Laptops) stores: '1/2/3/'
-- Querying all descendants of category_id=1 (Electronics, path '1/'):
SELECT * FROM categories WHERE materialized_path LIKE '1/%';
-- A simple LIKE prefix match, can use a B-tree index for the prefix
scan
-- (covered in SQL 3.7 – Indexes)
```

CONCEPT CARD

Decision rule - recursive CTE vs closure table vs materialized path:

code – Code Example

Use a recursive CTE when:

- The hierarchy changes frequently (recomputing a closure table on every change is wasteful)
- The query is run infrequently (admin tools, occasional reports)
- The hierarchy is shallow (under ~10 levels) and not too wide

Use a closure table when:

- The same hierarchy is queried very frequently (every page load, high-traffic API endpoint)
- The hierarchy changes infrequently relative to how often it is read
- You need fast arbitrary ancestor/descendant lookups in both directions

Use a materialized path column when:

- You mainly need 'is X a descendant of Y' or full-path display
- You want something simpler than maintaining a separate closure

table

- The hierarchy depth is bounded and path strings stay a manageable length

INTERVIEW CARD

Frequency: High

Question: A recursive CTE for a category breadcrumb is being called on every product page view and is showing up as a top query in your slow query log. What would you do?

Expected Answer: Recognise that the category hierarchy itself almost certainly changes far less often than it is read - recomputing the recursive traversal on every page view is wasted work. Replace the at-read-time recursive CTE with a precomputed structure: either a closure table that stores every ancestor-descendant pair with depth, refreshed whenever the category tree changes, or a simpler materialized path column on each category row updated on insert/move.

Both let the breadcrumb query become a simple indexed lookup or prefix match instead of a multi-iteration recursive JOIN, eliminating the recursive cost from the hot read path entirely. The recursive CTE itself is still useful - just relocated to the batch job that builds or refreshes the closure table, rather than running on every user-facing request.

Follow-up: What is the trade-off of a closure table compared to a materialized path column? (Answer: A closure table supports fast lookups in both directions - all ancestors of X, or all descendants of X - and handles complex cases like multiple paths to the same node more robustly, but requires more storage and more complex maintenance logic on every tree change.)

A materialized path column is simpler to maintain and sufficient for single-parent tree hierarchies and breadcrumb-style display, but is less flexible for graph-like structures with multiple parents.)

PRACTICE QUIZ

1. Why can a recursive CTE be expensive on a deep hierarchy?
2. What is a closure table and what does each row in it represent?
3. When is recomputing a closure table considered wasteful?
4. What SQL pattern lets you query descendants quickly using a materialized path column?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. Each level of depth requires a separate full execution of the recursive member's JOIN - a 15-level-deep hierarchy means 15 chained JOIN executions, with limited ability for the optimizer to optimize across the whole chain, and no index can speed up 'the result of iteration N' the way a normal WHERE clause benefits from an index.
2. A closure table precomputes and stores every ancestor-descendant pair (with depth) as flat rows, so ancestor/descendant lookups become simple indexed JOINS instead of recursive traversals at read time. Each row represents one (ancestorid, descendantid, depth) relationship.
3. When the hierarchy changes very frequently relative to how often it is queried - rebuilding the entire closure table on every small change can cost more than just running the recursive CTE directly when needed.
4. `WHERE materialized_path LIKE '1/%'` - a prefix match on the stored path string, which can use a B-tree index for an efficient prefix scan.

🔍 CHECKPOINT QUESTION

Your company's organisational chart changes roughly once a week (new hires, promotions) but the 'who reports to whom, at any depth' query is used in a permissions-check function called on nearly every API request - tens of thousands of times per day. Decide between a recursive CTE at read time, a closure table, or a materialized path column, and justify your choice including how you would keep it updated when the org chart changes.

PRACTICE PROBLEMS — UPWARD MANAGEMENT CHAIN AND DATE SEQUENCE GENERATION

ChatGPT • AI Assistant

A side-by-side comparison diagram of the same org chart traversed in two directions. Left panel (blue background, label 'WALKING DOWN — find descendants'): shows Carol at the top with arrows pointing DOWNWARD to Alice, Bob, Dave, and further down to Eve, Frank, Grace, Henry. The join condition box reads: 'e.manager_id = rc.employee_id — find children of the current row.' Starting point highlighted gold at Carol (top), recursion expands downward and outward. Right panel (orange background, label 'WALKING UP — find ancestors'): shows Henry at the top this time (as the starting point, highlighted gold) with arrows pointing UPWARD to Eve, then Alice, then Carol. The join condition box reads: 'e.employee_id = mc.manager_id — find the PARENT of the current row.' Recursion narrows upward to a single chain, terminating at Carol whose manager_id is NULL. A center divider box reads: 'Same anchor/UNION ALL/recursive member structure in both cases — only the JOIN condition direction flips between finding children (down) and finding the parent (up).' Clean side-by-side org-chart comparison, directional arrows clearly contrasted (downward branching tree vs upward single chain).

Downward vs Upward Recursion — Join Direction Comparison

Student

Let's work through the two practice problems.

ChatGPT • AI Assistant

CONCEPT CARD

Practice Problem 1 - Walk UP instead of down: return the full management chain ABOVE a given employee.

Requirement: given `employee_id = 8` (Henry), return Henry himself plus every manager above him, all the way to the CEO, with a level column where Henry is level 1 and each step up increases the level.

Solution:

sql – Code Example

```
WITH RECURSIVE management_chain AS (  
    -- ANCHOR: start at the GIVEN employee, not the CEO  
    SELECT employee_id,  
           name,  
           manager_id,  
           1 AS level  
    FROM   employees  
    WHERE  employee_id = 8           -- starting point: Henry  
  
    UNION ALL  
  
    -- RECURSIVE MEMBER: find the MANAGER of whoever was found  
    -- in the previous iteration – note the JOIN direction is REVERSED  
    -- compared to the downward example: here we join  
    employees.employee_id
```

```
-- to the PREVIOUS row's manager_id, walking UP the tree.
SELECT e.employee_id,
       e.name,
       e.manager_id,
       mc.level + 1
FROM   employees e
JOIN   management_chain mc ON e.employee_id = mc.manager_id
WHERE  mc.level < 50          -- depth guard for safety
)
SELECT employee_id, name, manager_id, level
FROM   management_chain
ORDER BY level;
```

Output (using the same sample data from earlier in this topic):

code – Code Example

employee_id	name	manager_id	level	
8	Henry	5	1	← starting employee
5	Eve	2	2	← Henry's manager
2	Alice	1	3	← Eve's manager
1	Carol	NULL	4	← Alice's manager (the CEO)

Recursion stops naturally at the CEO row, because Carol's `manager_id` is `NULL` , and the JOIN condition `e.employee_id = mc.manager_id` finds no employee whose `employee_id` equals `NULL` - zero rows, recursion halts.

The key structural difference from the downward example: the JOIN condition direction flips. Walking down: `e.manager_id = rc.employee_id` (find children of the current row).

Walking up: `e.employee_id = mc.manager_id` (find the parent of the current row). Everything else about the anatomy - anchor, UNION ALL, recursive member, depth guard - stays the same.

Practice Problem 2 - Generate a sequence of all dates between a start and end date, with no calendar table.

Requirement: produce one row per calendar day between `'2024-01-01'` and `'2024-01-10'` (inclusive), without creating or relying on a pre-existing dates table.

Solution:

sql – Code Example

```
WITH RECURSIVE date_sequence AS (  
    -- ANCHOR: the starting date itself  
    SELECT '2024-01-01'::DATE AS calendar_date  
  
    UNION ALL  
  
    -- RECURSIVE MEMBER: add one day to the PREVIOUS row's date,  
    -- stop once we would pass the end date  
    SELECT calendar_date + INTERVAL '1 day'  
    FROM   date_sequence  
    WHERE  calendar_date < '2024-01-10'::DATE  
)  
SELECT calendar_date  
FROM   date_sequence  
ORDER BY calendar_date;
```

Output:

code – Code Example

calendar_date

2024-01-01
2024-01-02
2024-01-03
2024-01-04
2024-01-05
2024-01-06
2024-01-07
2024-01-08
2024-01-09
2024-01-10

Notice this recursion has **no JOIN at all** in the recursive member - it does not need to reference another table, because each new row is generated purely from arithmetic on the previous row's single value. The `WHERE calendar_date < '2024-01-10'::DATE` clause inside the recursive member is itself the natural stopping condition; once `calendar_date` reaches `'2024-01-10'`, the WHERE clause is false, the recursive member produces zero rows, and recursion halts on its own - no separate depth guard is strictly required here since the date comparison already bounds it, though adding one is still good defensive practice for production code.

A practical use: joining the date sequence to orders to find days with ZERO orders (a gap analysis):

sql – Code Example

```
WITH RECURSIVE date_sequence AS (  
    SELECT MIN(order_date)::DATE AS calendar_date FROM orders  
    UNION ALL  
    SELECT calendar_date + INTERVAL '1 day'  
    FROM   date_sequence  
    WHERE  calendar_date < (SELECT MAX(order_date) FROM orders)  
)  
SELECT ds.calendar_date,  
       COUNT(o.order_id) AS orders_that_day  
FROM   date_sequence ds  
LEFT JOIN orders o ON o.order_date::DATE = ds.calendar_date  
GROUP BY ds.calendar_date  
HAVING COUNT(o.order_id) = 0           -- only days with NO orders  
ORDER BY ds.calendar_date;
```

This is the classic 'gap and island' problem solved cleanly: generate every possible day in the range first using a recursive CTE, then LEFT JOIN to the real data and filter for the days that produced no matches - revealing exactly which days had zero orders, something a plain JOIN or GROUP BY on the orders table alone could never show, since days with no orders simply have no rows to group in the first place.

SUMMARY CARD

SQL 1.12 Key Takeaways:

- **Recursive CTEs solve unknown-depth traversal problems** - hierarchies, trees, and sequences - that fixed-count JOINS cannot handle.
- **Syntax:** `WITH RECURSIVE cte_name AS (anchor_member UNION ALL recursive_member) .`
- **Anchor member runs once**, seeding the base case. **Recursive member runs repeatedly**, each time operating ONLY on the immediately preceding iteration's output - never the cumulative total, never re-running the anchor.
- Recursion **halts automatically** when the recursive member produces zero new rows across all active branches.
- **Walking down** a hierarchy: join `child.parent_col = cte.id_col` . **Walking up:** join `cte_table.id_col = cte.parent_col` - the join direction flips, everything else stays the same.
- **Always include a safeguard:** a depth guard (`WHERE level < N`) or a visited-path check (`WHERE NOT (id = ANY(visited_array))`) - cyclic data without a safeguard causes an infinite loop.

- Recursive CTEs **do not need a JOIN at all** for pure sequence generation (like date ranges) - pure arithmetic on the previous row is enough.
- **Performance reality:** each level of depth is a separate JOIN execution; deep or wide hierarchies can be expensive, and the optimizer cannot index 'across iterations.'
- **For frequently-read, infrequently-changed hierarchies**, move the recursive computation to a batch job that builds a **closure table** or **materialized path column**, and let the hot read path use simple indexed lookups instead.

🔮 AI DEEP DIVE PROMPT

Copy this prompt into Claude or ChatGPT for deeper practice:

'Act as a senior SQL mentor. I just learned recursive CTEs - the WITH RECURSIVE syntax, anchor and recursive members, walking hierarchies up and down, depth guards, visited-path cycle detection, and closure tables as a production alternative.

Give me 6 progressively harder exercises. Start with a simple downward hierarchy traversal, build up to a path-string builder, then a cycle-detection scenario, and finish with a recursive CTE used to generate a numeric sequence for a gap analysis.

For each exercise: give me the table schema and requirement, ask me to write the recursive CTE, then show the correct solution, trace through the first 2-3 iterations by hand, point out one common mistake, and give one interview question. Do not continue until I submit my answer.'

🔍 CHECKPOINT QUESTION

You need to find every ancestor category of a given product's category, but you also want to detect and flag (rather than crash on) any accidental cycle in the categories table during the same query. Write a recursive CTE using the visited-path technique that returns each ancestor found, and includes a boolean column `cycle_detected` that becomes TRUE the moment a node would be revisited, stopping further recursion on that branch immediately.